

Ain Shams - Faculty of Engineering

CSE485: Deep Learning

Hate Speech Detection Documentation

Team 1

Yehia Mahmoud Lotfy Sakr	21P0166
Marwan Nour-Eldin Farag	21P0165
Ahmed Ayman Aly	21P0002
Hana Mohamed Mohamed	21P0162
Youssef Ayman Mohamed Samy	21P0010

Contents

1. Project Overview 3
2. Dataset Description 3
3. Preprocessing and Data Cleaning 4
4. Model Architecture 5
5. Training Approaches 6
Approach 1: Train/Validation/Test Split with Class Weights 6
Approach 2: Synonym Augmentation (WordNet) 7
Approach 3: K-Fold Cross Validation with Weights 8
1. Stratified K-Fold Methodology 8
2. Dynamic Class Weighting 10
3. Model Training & Final Generalization 10
6. Results and Analysis 11
6.1 Augmentation Approach Results 11
6.2 Weights Approach Results 13
6.3 Cross-Validation Approach Results 15

1. Project Overview

This project focuses on the classification of social media posts (tweets) into three categories: **Hate Speech**, **Offensive Language**, and **Neither**. Given the unstructured nature of social media content, Deep Learning and Natural Language Processing (NLP) techniques are employed to filter out harmful content effectively.

2. Dataset Description

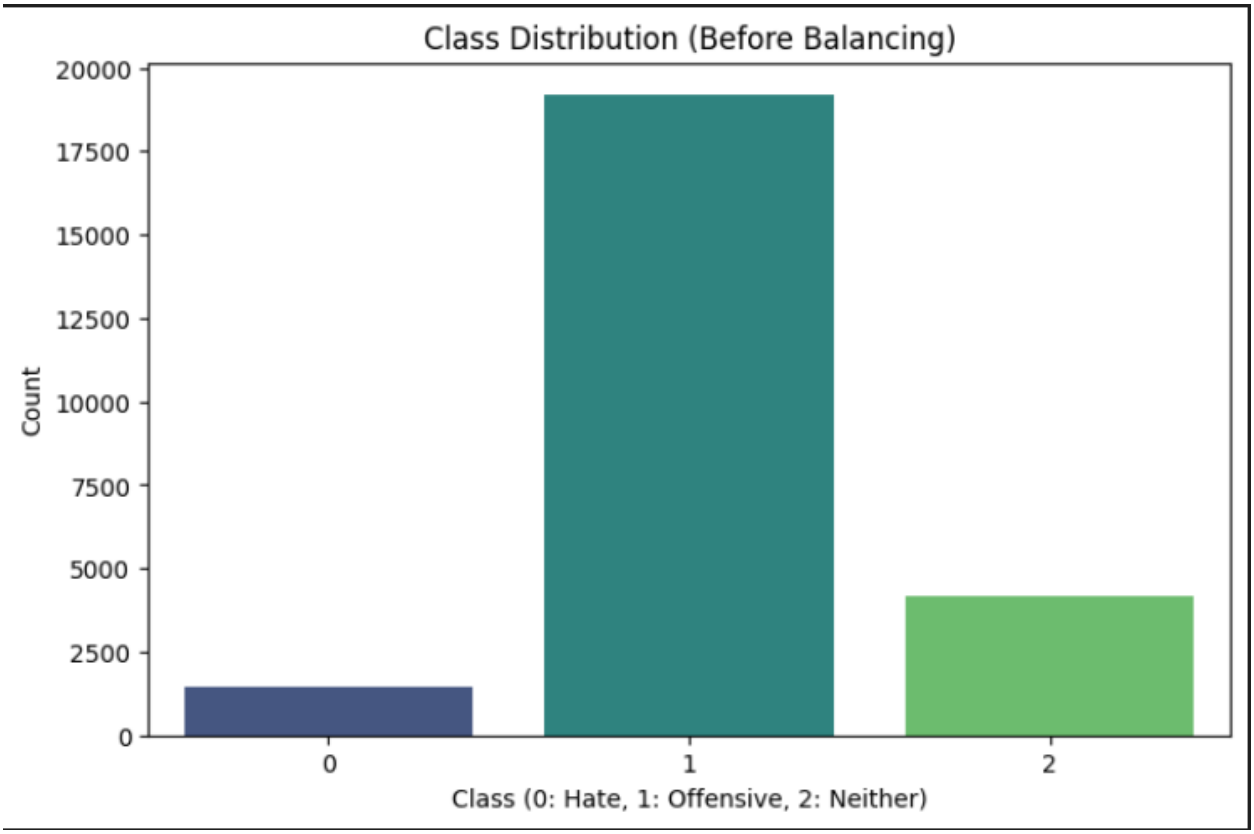
The dataset consists of **24,783 rows** with two primary columns: tweet (text) and class (label).

- **Class 0 (Hate Speech):** Content targeting individuals or groups with harmful intent.
- **Class 1 (Offensive Language):** Content containing offensive terms but not necessarily directed hate.
- **Class 2 (Neither):** Neutral or non-offensive content.

Data Imbalance

Initial exploration revealed a significant class imbalance:

- **Offensive Language (1):** ~19,190 samples (Majority class).
- **Neither (2):** ~4,163 samples.
- **Hate Speech (0):** ~1,430 samples (Minority class).



3. Preprocessing and Data Cleaning

To prepare the raw text for the Deep Learning model, the following steps were implemented:

1. **Text Normalization:** Converting all text to lowercase and removing special characters/punctuation using regular expressions.
2. **Stopwords Removal:** Removing common English words and Twitter-specific noise like "RT" and "amp".
3. **Stemming:** Using the **PorterStemmer** to reduce words to their base roots.
4. **Tokenization & Padding:** Converting cleaned text into numerical sequences with a maximum length of 100, utilizing an OOV (Out-Of-Vocabulary) token for unknown words.

```
twitter_stopwords = {'rt', 'amp'}

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
stop_words.update(twitter_stopwords)
stemmer = PorterStemmer()

# 1. Cleaning the Text
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^\w\s]', '', text)
    words = text.split()
    words = [stemmer.stem(w) for w in words if w not in stop_words]
    return " ".join(words)


df['clean_tweet'] = df['tweet'].apply(clean_text)

# 2. Train-Test-Val Split
X_train_full, X_test, y_train_full, y_test = train_test_split(
    df['clean_tweet'], df['class'], test_size=0.2, random_state=42, stratify=df['class']
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.2, random_state=42, stratify=y_train_full
)

# 3. Compute Class Weights
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)
class_weight_dict = dict(enumerate(class_weights))
print("Class Weights:", class_weight_dict)

# 4. Tokenization and Padding
MAX_WORDS = 5000
MAX_LEN = 100

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
```

4. Model Architecture

A Sequential Deep Learning model was built using **Keras/TensorFlow**:

- **Embedding Layer:** Maps words into a 128-dimensional dense vector space.
- **SpatialDropout1D:** Regularization layer (0.3) to prevent overfitting by dropping entire 1D feature maps.
- **Bidirectional LSTM:** A 64-unit Long Short-Term Memory layer that captures context from both the past and future directions of the sequence.
- **Dense Layers:** A hidden layer with 32 units (ReLU activation) followed by a final Softmax output layer for 3-class probability distribution.
- **Optimizer:** Adam optimizer with a learning rate of 0.0001.

```python
# 5. Build the Model
model = Sequential()
model.add(Embedding(input_dim=MAX_WORDS, output_dim=128))
model.add(SpatialDropout1D(0.3))
# Bidirectional LSTM helps capture context from both sides
model.add(Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.3)))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(3, activation='softmax'))

optimizer = Adam(learning_rate=0.0001)

model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# 6. Callbacks
# Reduce LR if stuck, Stop early if not improving
callbacks = [
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6, verbose=1),
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose=1)
]

# 7. Train the Model
print("\nStarting Training...")
history = model.fit(
    X_train_pad, y_train_enc,
    epochs=20,
    batch_size=32,
    validation_data=(X_val_pad, y_val_enc),
    class_weight=class_weight_dict,
    callbacks=callbacks,
    verbose=1
)
```

5. Training Approaches

The project explored three distinct methodologies to handle data imbalance and improve model robustness.

Approach 1: Train/Validation/Test Split with Class Weights

The dataset was split into training (64%), validation (16%), and testing (20%). To address the imbalance, **balanced class weights** were calculated using Scikit-learn, assigning a higher penalty to errors in the minority "Hate Speech" class (weight ~5.77) compared to the majority "Offensive" class (weight ~0.43).

```python
# 3. Compute Class Weights
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)
class_weight_dict = dict(enumerate(class_weights))
print("Class Weights:", class_weight_dict)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
Class Weights: {0: np.float64(5.777777777777778), 1: np.float64(0.4304752598865456), 2: np.float64(1.9844844844844844)}
```

Approach 2: Synonym Augmentation (WordNet)

This approach balanced the dataset before training by generating new samples for minority classes.

- **Method:** Words in existing tweets were randomly replaced with their synonyms retrieved from the **WordNet** database (wordnet.synsets).
- **Result:** Classes 0 and 2 were augmented to match the majority class size (~12,281 samples each), resulting in a perfectly balanced training set of 36,843 samples.

```python
# 3. Data Augmentation to Balance Classes
print(f"Original Training Shape: {X_train.shape}")
print("Class distribution before augmentation:\n", y_train.value_counts())

def get_synonyms(word):
    """Get a set of synonyms for a word from WordNet."""
    synonyms = set()
    for syn in wordnet.synsets(word):
        for lemma in syn.lemmas():
            candidate = lemma.name().replace("_", " ").lower()
            if candidate != word and len(candidate.split()) == 1: # Avoid phrases
                synonyms.add(candidate)
    return list(synonyms)

def augment_text(text, num_replacements=1):
    """Replace 'num_replacements' words in the text with synonyms."""
    words = text.split()
    if len(words) < 3: return text # Skip very short tweets

    new_words = words.copy()
    random_indices = random.sample(range(len(words)), min(len(words), num_replacements))

    for idx in random_indices:
        word = words[idx]
        syns = get_synonyms(word)
        if syns:
            new_words[idx] = random.choice(syns)

    return " ".join(new_words)
```

```
Original Training Shape: (15860,)
Class distribution before augmentation:
 class
1    12281
2     2664
0      915
Name: count, dtype: int64

Starting Augmentation (this may take a moment)...
  -> Class 1 is the majority (12281). No augmentation needed.
  -> Augmenting Class 0: Generating 11366 new samples...
  -> Augmenting Class 2: Generating 9617 new samples...

Final Training Shape: (36843,)
Class distribution after augmentation:
 label
0    12281
1    12281
2    12281
Name: count, dtype: int64
```

Approach 3: K-Fold Cross Validation with Weights

This approach combines a robust statistical evaluation method with cost-sensitive learning to address both class imbalance and potential data-split bias. The process is divided into two phases: cross-validation for stability assessment and final evaluation on held-out data.

1. Stratified K-Fold Methodology

To ensure the model's performance is consistent across different subsets of the data, we implemented a **5-fold Stratified Cross-Validation** on 80% of the dataset (the Development Set).

- **Stratification**: Unlike standard K-fold, the "Stratified" approach ensures that each of the 5 folds maintains the original percentage of samples for each of the three classes (Hate, Offensive, and Neither). This is critical for imbalanced datasets to prevent folds from lacking minority class representation.
- **Iterative Validation**: The model is trained five times. In each iteration, 4 folds are used for training and 1 fold is held back for validation. This guarantees that every data point in the development set is used for validation exactly once.

```
ontrol (Ctrl+Shift+G)
S-VALIDATION ON DEVELOPMENT SET ---
# We only use X_dev and y_dev here
kfold = StratifiedKFold(n_splits=FOLDS, shuffle=True, random_state=42)
cv_accuracies = []
fold_no = 1

print(f"\nStarting {FOLDS}-Fold Stratified Cross-Validation on DEVELOPMENT SET...\n")

for train_index, val_index in kfold.split(X_dev, y_dev):
    print(f"--- FOLD {fold_no} / {FOLDS} ---")

    # Split Dev data into Train/Val for this fold
    X_fold_train, X_fold_val = X_dev[train_index], X_dev[val_index]
    y_fold_train, y_fold_val = y_dev[train_index], y_dev[val_index]

    # Compute Class Weights
    fold_weights = class_weight.compute_class_weight(
        class_weight='balanced',
        classes=np.unique(y_fold_train),
        y=y_fold_train
    )
    class_weight_dict = dict(enumerate(fold_weights))

    # Tokenize (Fit only on fold training data)
    tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
    tokenizer.fit_on_texts(X_fold_train)

    X_train_seq = tokenizer.texts_to_sequences(X_fold_train)
    X_val_seq = tokenizer.texts_to_sequences(X_fold_val)
    X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='pre', truncating='post')
    X_val_pad = pad_sequences(X_val_seq, maxlen=MAX_LEN, padding='pre', truncating='post')

    y_train_enc = to_categorical(y_fold_train, num_classes=3)
    y_val_enc = to_categorical(y_fold_val, num_classes=3)

    # Build Model
    clear_session()
    model = Sequential()
    model.add(Embedding(input_dim=MAX_WORDS, output_dim=128))
    model.add(SpatialDropout1D(0.3))
    model.add(Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.3)))
    model.add(Dense(32, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(3, activation='softmax'))

    optimizer = Adam(learning_rate=0.0001)
    model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])

    # Train
    es = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True, verbose=0)
    model.fit(
```

2. Dynamic Class Weighting

During each of the five training folds, **class weights** are recalculated using the balanced heuristic. This ensures the model penalizes errors on the minority "Hate Speech" and "Neither" classes more heavily than on the majority "Offensive Language" class. This allows the Bidirectional LSTM to learn the distinguishing features of less frequent labels without the need for oversampling or synthetic data generation.

```
# Compute Class Weights
fold_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_fold_train),
    y=y_fold_train
)
class_weight_dict = dict(enumerate(fold_weights))
```

3. Model Training & Final Generalization

- **Architecture**: Each fold utilizes a **Bidirectional LSTM** with **SpatialDropout1D (0.3)** and an **Adam optimizer**.
- **Robustness Check**: By calculating the mean accuracy and standard deviation across all 5 folds, we obtain a reliable estimate of the model's expected performance on unseen data, effectively eliminating the risk of reporting "lucky" results from a single train-test split.
- **Held-out Evaluation**: After confirming stability through cross-validation, a final model is trained on the entire development set and evaluated on a separate **20% held-out test set** to report the final classification metrics (Precision, Recall, and F1-score).

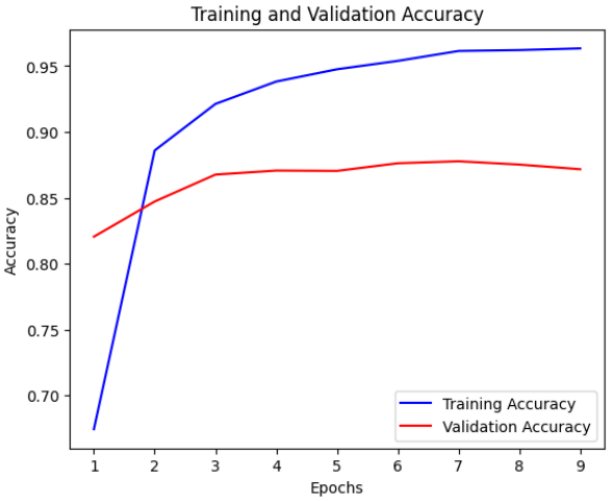
6. Results and Analysis

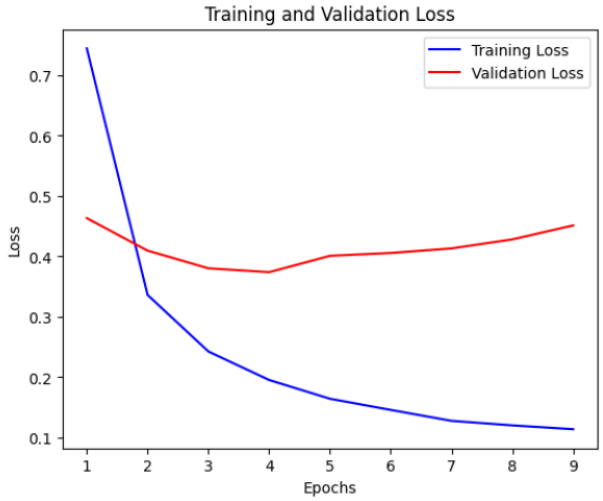
- **Callback Performance:** Both EarlyStopping and ReduceLROnPlateau were effective in preventing overfitting and fine-tuning the learning rate when validation loss plateaued.

6.1 Augmentation Approach Results

- **Augmentation Impact:** Approach 2 reached a high training accuracy (~96%) but showed early signs of validation loss increase, suggesting the model began memorizing the synthetic patterns.

```
Starting Training with Augmented Data...
Epoch 1/20
1152/1152 ━━━━━━━━━━━━━━━━━━━━ 675s 567ms/step - accuracy: 0.5283 - loss: 0.9491 - val_accuracy: 0.8205 - val_loss: 0.4633 - learning_rate: 1.0000e-04
Epoch 2/20
...
Epoch 9/20
1152/1152 ━━━━━━━━━━━━━━━━━━━━ 311s 270ms/step - accuracy: 0.9614 - loss: 0.1176 - val_accuracy: 0.8717 - val_loss: 0.4512 - learning_rate: 2.5000e-05
Epoch 9: early stopping
Restoring model weights from the end of the best epoch: 4.
```



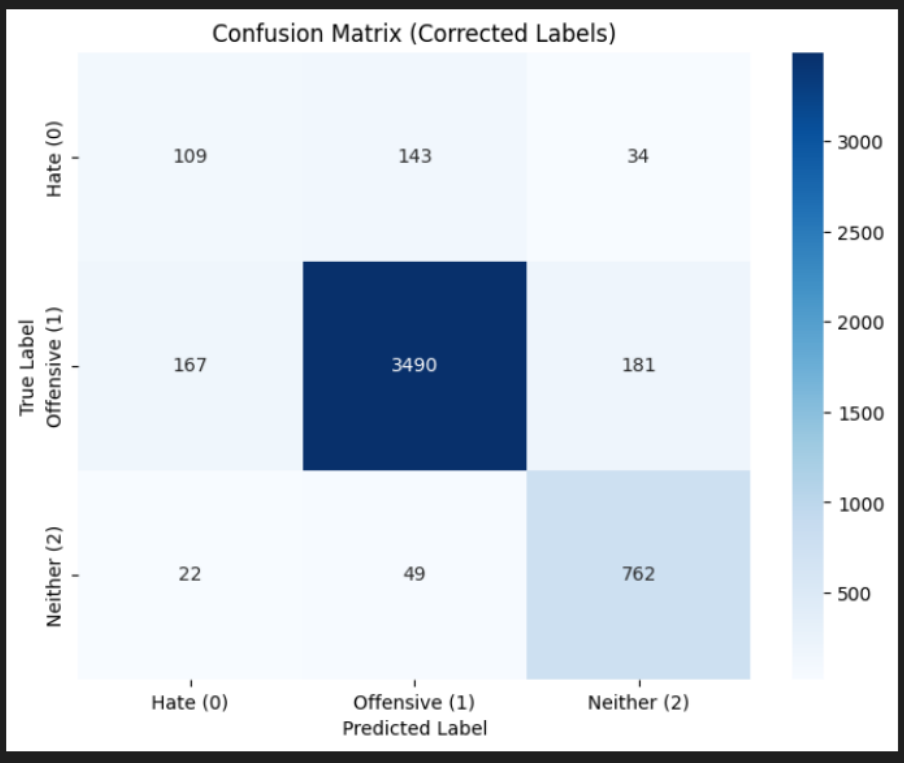


```
Classification Report:

              precision    recall  f1-score   support

     Hate (0)       0.37      0.38      0.37       286
Offensive (1)       0.95      0.91      0.93      3838
  Neither (2)       0.78      0.91      0.84       833

     accuracy                           0.88      4957
    macro avg       0.70      0.74      0.71      4957
 weighted avg       0.89      0.88      0.88      4957
```

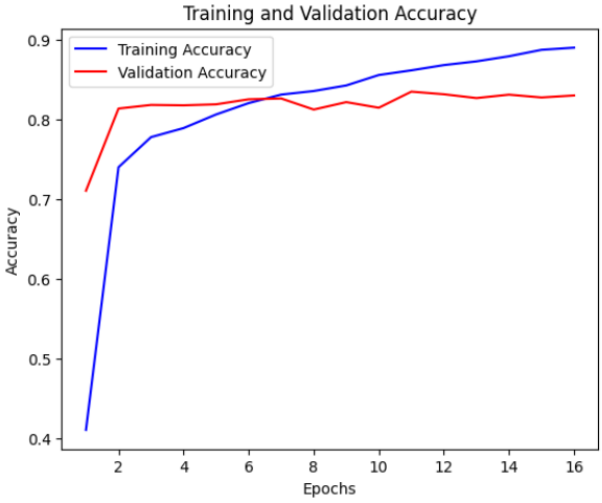


‘

6.2 Weights Approach Results

- **Weighting Impact:** Approach 1 achieved more stable validation accuracy (~83%) by focusing the model's attention on correctly identifying the rare Hate Speech samples without artificial data generation.

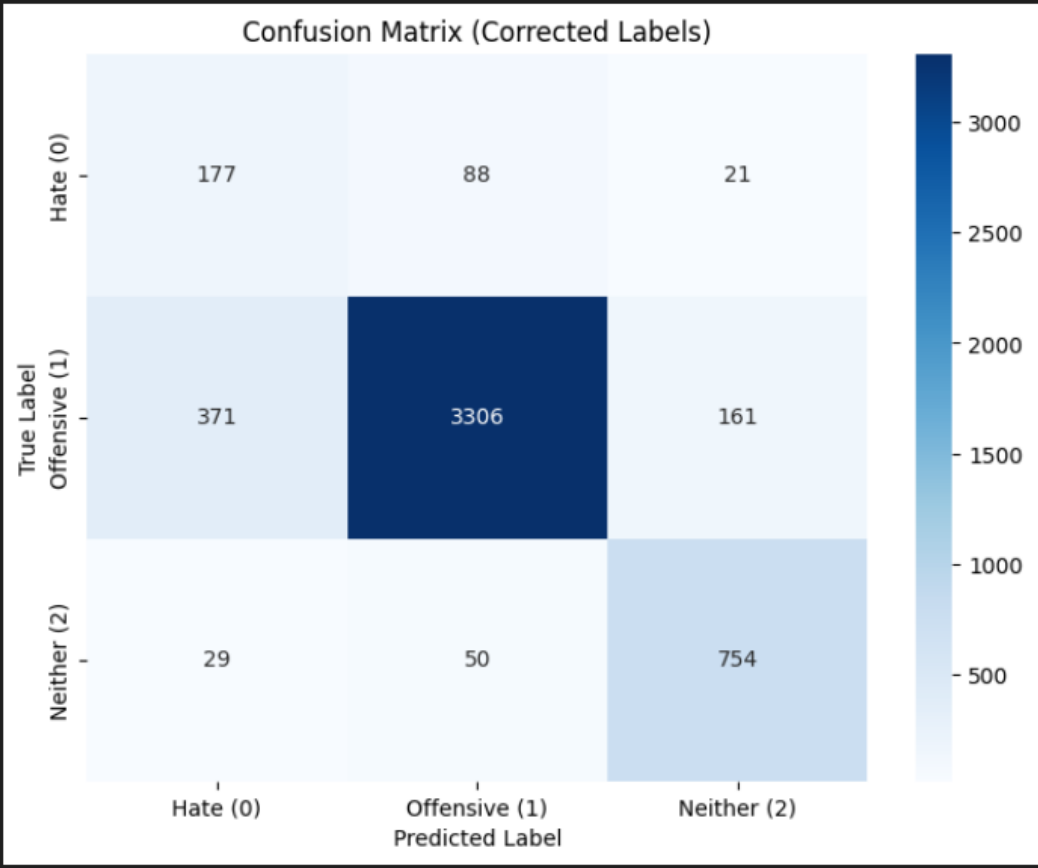
```
Starting Training...
Epoch 1/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 141s 268ms/step - accuracy: 0.3439 - loss: 1.0799 - val_accuracy: 0.7103 - val_loss: 1.0000 - learning_rate: 1.0000e-04
Epoch 2/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 140s 263ms/step - accuracy: 0.7139 - loss: 0.9297 - val_accuracy: 0.8134 - val_loss: 0.5997 - learning_rate: 1.0000e-04
Epoch 3/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 131s 265ms/step - accuracy: 0.7878 - loss: 0.6907 - val_accuracy: 0.8180 - val_loss: 0.5905 - learning_rate: 1.0000e-04
Epoch 4/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 143s 268ms/step - accuracy: 0.7799 - loss: 0.6146 - val_accuracy: 0.8174 - val_loss: 0.5632 - learning_rate: 1.0000e-04
Epoch 5/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 146s 276ms/step - accuracy: 0.8023 - loss: 0.5397 - val_accuracy: 0.8187 - val_loss: 0.5398 - learning_rate: 1.0000e-04
Epoch 6/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 137s 266ms/step - accuracy: 0.8204 - loss: 0.4930 - val_accuracy: 0.8250 - val_loss: 0.4975 - learning_rate: 1.0000e-04
Epoch 7/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 145s 273ms/step - accuracy: 0.8331 - loss: 0.4597 - val_accuracy: 0.8260 - val_loss: 0.4848 - learning_rate: 1.0000e-04
Epoch 8/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 142s 274ms/step - accuracy: 0.8347 - loss: 0.4068 - val_accuracy: 0.8122 - val_loss: 0.5102 - learning_rate: 1.0000e-04
Epoch 9/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 137s 277ms/step - accuracy: 0.8386 - loss: 0.3800 - val_accuracy: 0.8215 - val_loss: 0.4824 - learning_rate: 1.0000e-04
Epoch 10/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 132s 266ms/step - accuracy: 0.8588 - loss: 0.3439 - val_accuracy: 0.8144 - val_loss: 0.5023 - learning_rate: 1.0000e-04
Epoch 11/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 134s 270ms/step - accuracy: 0.8610 - loss: 0.3314 - val_accuracy: 0.8346 - val_loss: 0.4466 - learning_rate: 1.0000e-04
...
Epoch 16/20
496/496 ━━━━━━━━━━━━━━━━━━━━ 130s 262ms/step - accuracy: 0.8870 - loss: 0.2706 - val_accuracy: 0.8298 - val_loss: 0.4803 - learning_rate: 2.5000e-05
Epoch 16: early stopping
Restoring model weights from the end of the best epoch: 11.
```





Classification Report:

	precision	recall	f1-score	support
Hate (0)	0.31	0.62	0.41	286
Offensive (1)	0.96	0.86	0.91	3838
Neither (2)	0.81	0.91	0.85	833
accuracy			0.85	4957
macro avg	0.69	0.80	0.72	4957
weighted avg	0.90	0.85	0.87	4957



6.3 Cross-Validation Approach Results

- **Cross-Validation Robustness:** Approach 3 provided the most statistically sound evaluation by integrating Stratified K-Fold with class weighting. By averaging performance across five distinct data slices, this method mitigated the risk of "split-luck" (where a single test set might be unusually easy or hard). The narrow standard deviation across folds confirmed that the model's ability to generalize to minority classes was stable and not dependent on a specific subset of the data.

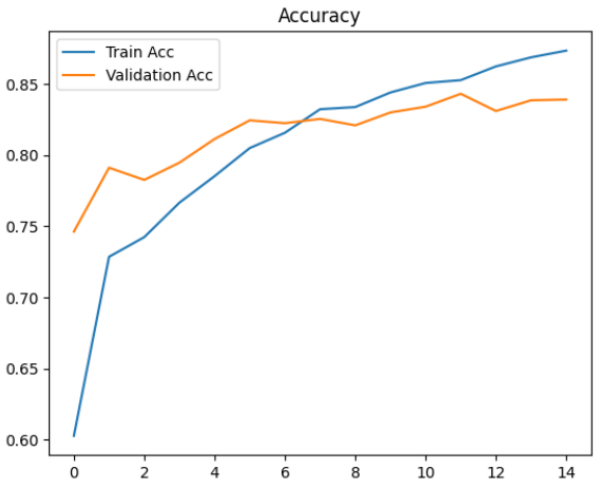
```
Starting 5-Fold Stratified Cross-Validation on DEVELOPMENT SET...

--- FOLD 1 / 5 ---
Fold 1 Accuracy: 82.63%
--- FOLD 2 / 5 ---
Fold 2 Accuracy: 82.65%
--- FOLD 3 / 5 ---
Fold 3 Accuracy: 83.88%
--- FOLD 4 / 5 ---
Fold 4 Accuracy: 84.56%
--- FOLD 5 / 5 ---
Fold 5 Accuracy: 84.69%

============================
CROSS VALIDATION RESULTS
============================
Individual Scores: ['82.63%', '82.65%', '83.88%', '84.56%', '84.69%']
Mean Accuracy:     83.68%
Standard Dev:      0.90%
============================
```

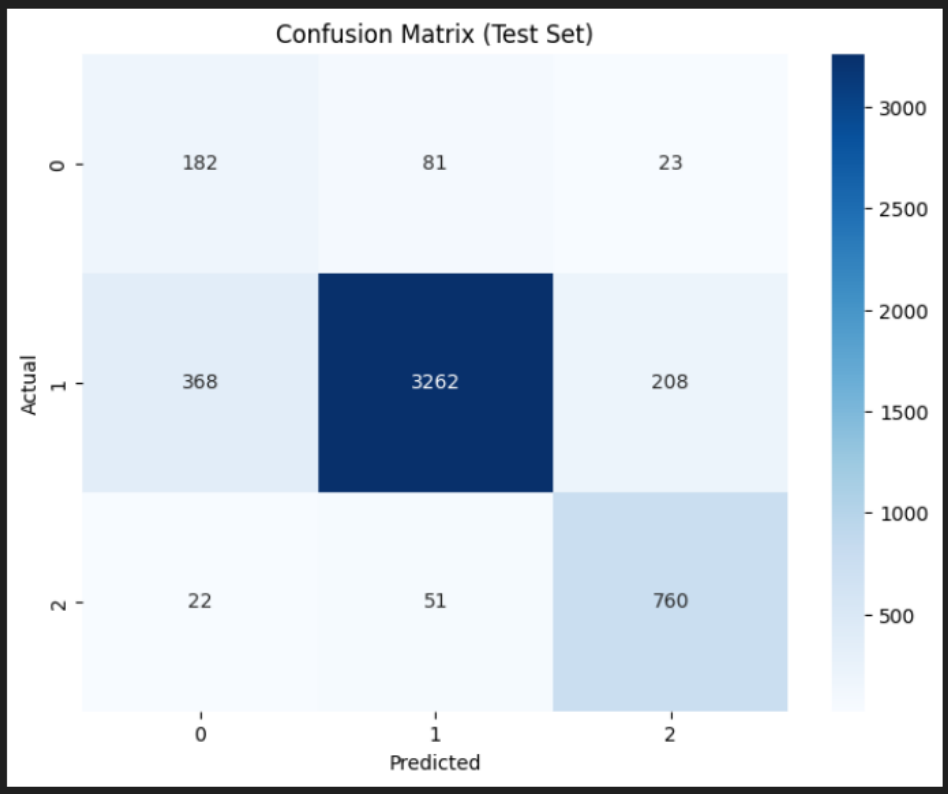
```
Epoch 1/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 163s 276ms/step - accuracy: 0.5556 - loss: 1.1053 - val_accuracy: 0.7463 - val_loss: 0.9326
Epoch 2/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 151s 271ms/step - accuracy: 0.7207 - loss: 0.9310 - val_accuracy: 0.7912 - val_loss: 0.6316
Epoch 3/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 206s 278ms/step - accuracy: 0.7510 - loss: 0.6739 - val_accuracy: 0.7827 - val_loss: 0.5955
Epoch 4/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 165s 295ms/step - accuracy: 0.7624 - loss: 0.5887 - val_accuracy: 0.7948 - val_loss: 0.5740
Epoch 5/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 155s 279ms/step - accuracy: 0.7678 - loss: 0.5724 - val_accuracy: 0.8114 - val_loss: 0.5283
Epoch 6/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 204s 282ms/step - accuracy: 0.7926 - loss: 0.5008 - val_accuracy: 0.8245 - val_loss: 0.4760
Epoch 7/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 153s 274ms/step - accuracy: 0.8174 - loss: 0.4546 - val_accuracy: 0.8225 - val_loss: 0.4713
Epoch 8/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 157s 282ms/step - accuracy: 0.8317 - loss: 0.4296 - val_accuracy: 0.8255 - val_loss: 0.4519
Epoch 9/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 200s 279ms/step - accuracy: 0.8323 - loss: 0.3863 - val_accuracy: 0.8210 - val_loss: 0.4682
Epoch 10/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 157s 282ms/step - accuracy: 0.8387 - loss: 0.3621 - val_accuracy: 0.8301 - val_loss: 0.4426
Epoch 11/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 154s 275ms/step - accuracy: 0.8485 - loss: 0.3535 - val_accuracy: 0.8341 - val_loss: 0.4476
Epoch 12/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 203s 277ms/step - accuracy: 0.8522 - loss: 0.3216 - val_accuracy: 0.8432 - val_loss: 0.4114
Epoch 13/15
...
Epoch 15/15
558/558 ━━━━━━━━━━━━━━━━━━━━ 204s 285ms/step - accuracy: 0.8729 - loss: 0.2736 - val_accuracy: 0.8391 - val_loss: 0.4356

Saving artifacts...
```





```
FINAL TEST SET EVALUATION
============================
155/155 ━━━━━━━━━━━━━━━━━━━━ 10s 57ms/step
```



	precision	recall	f1-score	support
0	0.32	0.64	0.42	286
1	0.96	0.85	0.90	3838
2	0.77	0.91	0.83	833
accuracy			0.85	4957
macro avg	0.68	0.80	0.72	4957
weighted avg	0.89	0.85	0.86	4957